

```
"""
```

```
fase1_univariado.py
```

```
=====
```

FASE 1 - Pruebas univariadas.

Evalua CADA columna del dataset (las de rol se listan igualmente, marcadas, para que la tabla sea la fotografia completa) y aplica dos criterios de eliminacion que no requieren mirar el target:

1.1 **Exceso de ceros + nulos**

Una variable cuyo (%ceros + %nulos) supera el umbral es, en la practica, una constante con excepciones. Aporta senal en una fraccion minima de la muestra y en panel esa fraccion suele concentrarse en unas pocas entidades, con lo que el modelo aprenderia el identificador y no el fenomeno. Umbral principal 95%; alterno conservador 90%.

1.2 **Baja variacion**

Una variable sin dispersion no puede explicar la dispersion del target. Se aplican cuatro pruebas complementarias porque ninguna basta sola:

- varianza / desviacion estandar ~ 0 -> constante en escala absoluta
- coeficiente de variacion ~ 0 -> constante en escala relativa
- una categoria domina $\geq 99\%$ de la masa -> casi constante (sirve para numericas y categoricas por igual)
- IQR ~ 0 y percentiles comprimidos -> la distribucion colapsa en un punto aunque tenga colas extremas

Nada se elimina sin dejar el motivo escrito: cada flag va acompañado de una columna de texto con la razon exacta.

```
"""
```

```
from __future__ import annotations
```

```
import numpy as np
```

```
import pandas as pd
```

```
from .config import ConfigPipeline
```

```
from .logging_utils import obtener_logger
```

```
LOGGER = obtener_logger("fase1")
```

```
def _evaluar_ceros_nulos(fila: pd.Series, cfg: ConfigPipeline) -> tuple[int, str]:
    """Criterio 1.1. Devuelve (flag, motivo)."""
    umbral = cfg.umbral_ceros_nulos_efectivo
    pct = fila.get("pct_ceros_mas_nulos", np.nan)
    if pd.isna(pct):
        return 0, ""
    if pct >= umbral:
        etiqueta = "alterno conservador" if cfg.usar_umbral_alterno else "principal"
        return 1, (
            f"ceros+nulos={pct:.2%} >= umbral {etiqueta} {umbral:.0%} "
            f"(nulos={fila['pct_nulos']:.2%}, ceros={fila['pct_ceros']:.2%})"
        )
    return 0, ""
```

```
def _evaluar_variacion(fila: pd.Series, cfg: ConfigPipeline) -> tuple[int, str, dict]:
    """Criterio 1.2. Devuelve (flag, motivo, subflags)."""
    sub = {
        "flg_constante": 0,
        "flg_std_baja": 0,
        "flg_cv_bajo": 0,
        "flg_categoria_dominante": 0,
        "flg_percentiles_comprimidos": 0,
    }
    motivos: list[str] = []

    # (a) Un solo valor distinto: constante pura.
    if fila["n_unicos"] < cfg.minimo_valores_unicos:
```

```

sub["flg_constante"] = 1
motivos.append(f"solo {fila['n_unicos']} valor(es) unico(s)")

# (b) Desviacion estandar practicamente nula (escala absoluta).
std = fila.get("desviacion_estandar", np.nan)
if pd.notna(std) and std <= cfg.umbral_std_minimo:
    sub["flg_std_baja"] = 1
    motivos.append(f"desviacion estandar={std:.3e} <= {cfg.umbral_std_minimo:.1e}")

# (c) Coeficiente de variacion despreciable (escala relativa).
cv = fila.get("coef_variacion", np.nan)
if pd.notna(cv) and cv <= cfg.umbral_cv_minimo:
    sub["flg_cv_bajo"] = 1
    motivos.append(f"coef. de variacion={cv:.3e} <= {cfg.umbral_cv_minimo:.1e}")

# (d) Una sola categoria/valor concentra casi toda la muestra.
dom = fila.get("pct_valor_mas_frecuente", np.nan)
if pd.notna(dom) and dom >= cfg.umbral_dominancia:
    sub["flg_categoria_dominante"] = 1
    motivos.append(
        f"el valor '{fila['valor_mas_frecuente']}' cubre {dom:.2%} "
        f">= {cfg.umbral_dominancia:.0%} de los no nulos"
    )

# (e) Percentiles comprimidos: p25 = p99 significa que al menos el 74% de
#     la distribucion esta en un unico punto. El IQR nulo por si solo no
#     alcanza (es normal en conteos), por eso se exige tambien el p99.
p25, p75, p99 = fila.get("p25", np.nan), fila.get("p75", np.nan), fila.get("p99", np.nan)
if pd.notna(p25) and pd.notna(p75) and pd.notna(p99):
    iqr = p75 - p25
    if iqr <= cfg.umbral_iqr_minimo and np.isclose(p99, p25, rtol=1e-9, atol=1e-12):
        sub["flg_percentiles_comprimidos"] = 1
        motivos.append(f"percentiles comprimidos (p25=p75=p99={p25:g}, IQR={iqr:g})")

flag = int(any(sub.values()))
return flag, "; ".join(motivos), sub

```

```

def ejecutar(diagnostico: pd.DataFrame, cfg: ConfigPipeline) -> pd.DataFrame:
    """Ejecuta la fase univariada sobre la tabla de diagnostico.

```

Parameters

diagnostico

Salida de la fase 0 (una fila por columna del dataset original).

Returns

pd.DataFrame

Tabla univariada con TODAS las columnas originales y sus flags.

Columnas minimas exigidas por la especificacion: ``columna``,
 ``pct\_ceros+nulos``, ``desviacion\_estandar``, ``varianza``, ``p25``,
 ``p50``, ``p75``, ``p90``, ``p99``, ``flg\_eliminado\_ceros``,
 ``flg\_eliminado\_variacion``, ``flg\_seleccionada\_univariada``.

"""

```

LOGGER.info("=" * 78)

```

```

LOGGER.info("FASE 1 - PRUEBAS UNIVARIADAS")

```

```

LOGGER.info(

```

```

    "Umbral ceros+nulos = %.0f%% (%s) | std_min=%.1e | cv_min=%.1e | dominancia=%.0f%%",
    cfg.umbral_ceros_nulos_efectivo * 100,
    "alterno conservador" if cfg.usar_umbral_alterno else "principal",
    cfg.umbral_std_minimo, cfg.umbral_cv_minimo, cfg.umbral_dominancia * 100,

```

```

)

```

```

LOGGER.info("=" * 78)

```

```

base_cols = [

```

```

    "columna", "rol", "tipo_inferido", "n_filas", "n_nulos", "pct_nulos",
    "n_ceros", "pct_ceros", "pct_ceros_mas_nulos", "n_unicos", "pct_unicos",
    "valor_mas_frecuente", "pct_valor_mas_frecuente", "media",
    "desviacion_estandar", "varianza", "coef_variacion", "minimo", "maximo",
    "p25", "p50", "p75", "p90", "p99", "iqr", "rango", "asimetria", "curtosis",

```

```

    "var_within", "var_between", "icc_panel",
]
uni = diagnostico[[c for c in base_cols if c in diagnostico.columns]].copy()

filas_flags = []
for _, fila in uni.iterrows():
    es_candidata = fila["rol"] == "CANDIDATA"

    if not es_candidata:
        # Las columnas de rol se listan para completitud, nunca se
        # "eliminan" (no participaban de la seleccion).
        filas_flags.append(
            {
                "flg_eliminado_ceros": 0, "flg_eliminado_variacion": 0,
                "flg_seleccionada_univariada": 0,
                "flg_constante": 0, "flg_std_baja": 0, "flg_cv_bajo": 0,
                "flg_categoria_dominante": 0, "flg_percentiles_comprimidos": 0,
                "motivo_ceros": "", "motivo_variacion": "",
                "decision_univariada": f"NO_APLICA ({fila['rol']})",
            }
        )
        continue

    f_ceros, m_ceros = _evaluar_ceros_nulos(fila, cfg)
    f_var, m_var, sub = _evaluar_variacion(fila, cfg)

    seleccionada = int(f_ceros == 0 and f_var == 0)
    if seleccionada:
        decision = "RETENIDA"
    elif f_ceros and f_var:
        decision = "ELIMINADA_CEROS_Y_VARIACION"
    elif f_ceros:
        decision = "ELIMINADA_CEROS_NULOS"
    else:
        decision = "ELIMINADA_BAJA_VARIACION"

    filas_flags.append(
        {
            "flg_eliminado_ceros": f_ceros,
            "flg_eliminado_variacion": f_var,
            "flg_seleccionada_univariada": seleccionada,
            **sub,
            "motivo_ceros": m_ceros,
            "motivo_variacion": m_var,
            "decision_univariada": decision,
        }
    )

uni = pd.concat([uni.reset_index(drop=True), pd.DataFrame(filas_flags)], axis=1)

# Alias exigido literalmente por la especificacion de la salida.
uni["pct_ceros+nulos"] = uni["pct_ceros_mas_nulos"]

# Orden: primero las retenidas, luego las eliminadas, luego los roles.
orden = {"RETENIDA": 0, "ELIMINADA_CEROS_NULOS": 1, "ELIMINADA_BAJA_VARIACION": 2,
         "ELIMINADA_CEROS_Y_VARIACION": 3}
uni["_orden"] = uni["decision_univariada"].map(orden).fillna(9)
uni = uni.sort_values(["_orden", "columna"]).drop(columns="_orden").reset_index(drop=True)

n_cand = int((uni["rol"] == "CANDIDATA").sum())
n_ceros = int(uni["flg_eliminado_ceros"].sum())
n_var = int(uni["flg_eliminado_variacion"].sum())
n_sel = int(uni["flg_seleccionada_univariada"].sum())

LOGGER.info("Candidatas evaluadas          : %d", n_cand)
LOGGER.info("Eliminadas por ceros+nulos      : %d", n_ceros)
LOGGER.info("Eliminadas por baja variacion : %d", n_var)
LOGGER.info("Sobrevivientes de la fase 1   : %d (%.1f%% de las candidatas)",
           n_sel, 100 * n_sel / n_cand if n_cand else 0)

for _, f in uni[(uni["rol"] == "CANDIDATA") & (uni["flg_seleccionada_univariada"] == 0)].iterrows():

```

```

        LOGGER.debug("    - %s -> %s | %s %s", f["columna"], f["decision_univariada"],
                    f["motivo_ceros"], f["motivo_variacion"])

    if n_sel == 0:
        LOGGER.error(
            "Ninguna variable sobrevivio la fase univariada. Revise los umbrales "
            "(umbral_ceros_nulos=%.2f, umbral_dominancia=%.2f) o la calidad del dataset.",
            cfg.umbral_ceros_nulos_efectivo, cfg.umbral_dominancia,
        )

    return uni

def obtener_sobrevivientes(uni: pd.DataFrame) -> list[str]:
    """Lista de variables que pasan a la fase bivariada."""
    return uni.loc[uni["flg_seleccionada_univariada"] == 1, "columna"].tolist()

```